Number Representation and Data Structures

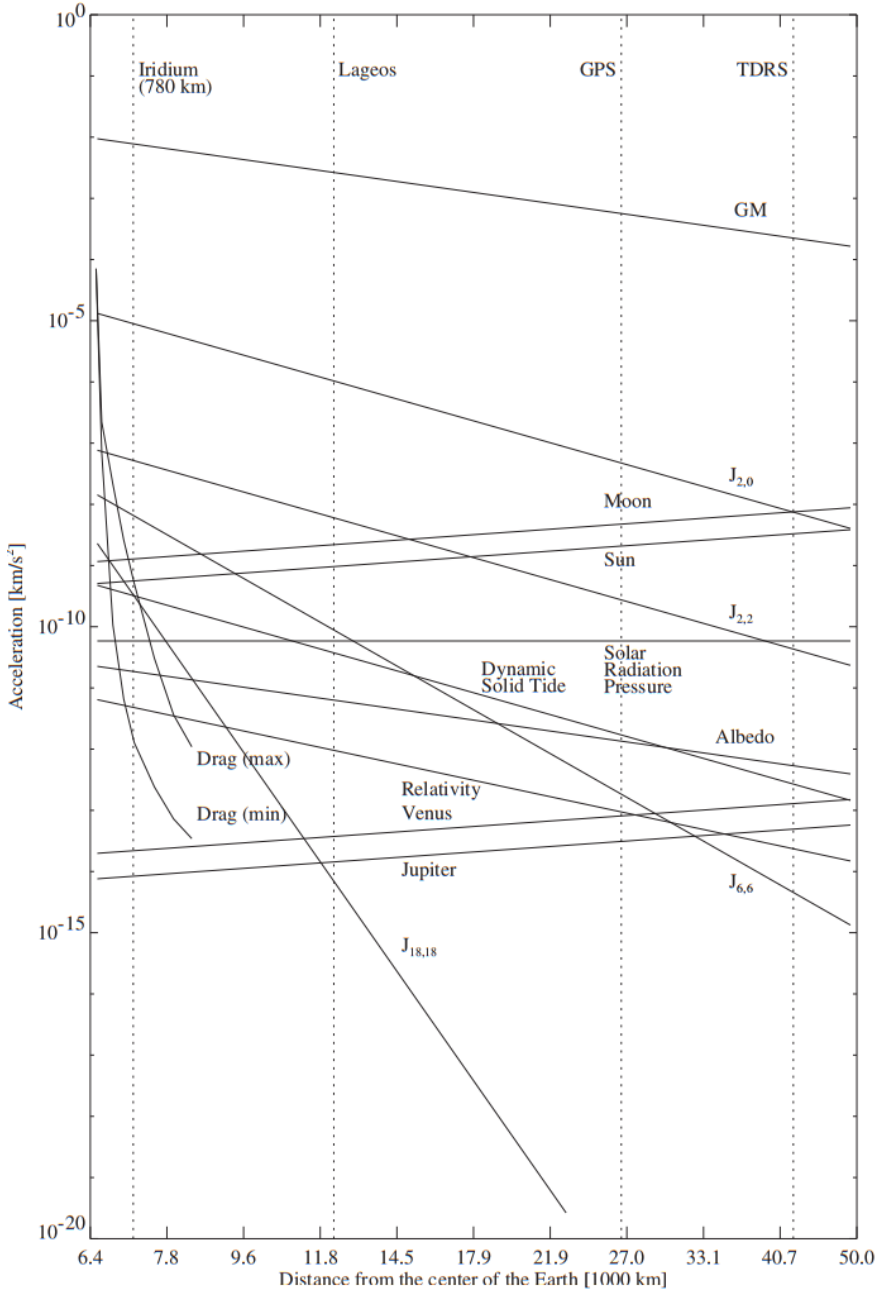
Elizabeth Klioner M.Sc., Niklas Meyners M.Sc. | Chair of Space Technology | Institute of Aeronautics and Astronautics (ILR)

Number Representation (1)

- Computers are binary, how are numbers in the decimal system represented?
  - Integers
    - » Signed
    - » Unsigned
  - Fixed Precision Numbers
  - Floating-Point Numbers
- How accurate is the Floating-Point Number representation?
  - Depends on the size (32 bit, 64 bit, etc.)

Number Representation (2)

- Order of magnitude of values encountered in orbit simulations can differ significantly, e.g.:
  - Distance Earth-Sun
  - Atmospheric Drag acceleration
  - Solar Radiation Pressure acceleration
- Are arithmetic operations (addition, subtraction, multiplication, division) exact floating point numbers?
  - Not always, errors can occur



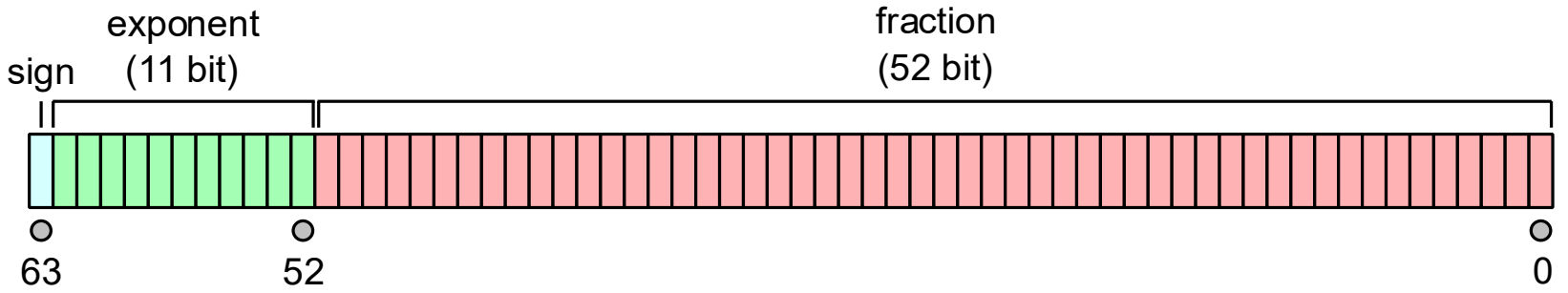
[1]

Floating-Point Numbers

- Number representation similar to scientific notation (i.e.: $1.234 \cdot 10^5$)
- Standardized in IEEE 754, almost all systems and programming languages adhere to this standard
- Most common standard types of floating-point numbers are:
  - single-precision (*single*) with a size of 32 bits
  - double-precision (*double*) with a size of 64 bits
- For orbit simulations *doubles* are used regularly

IEEE 754 Double-Precision Floating-Point Numbers (1)

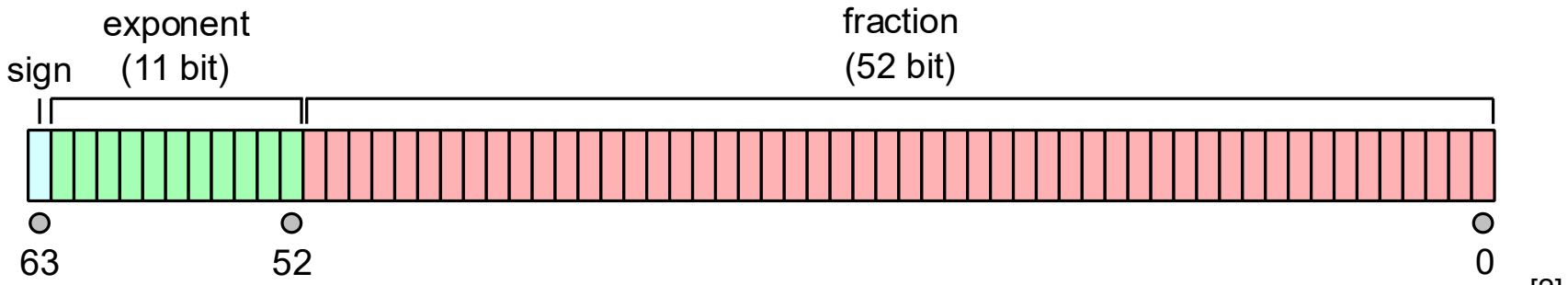
- Double-precision floating-point numbers have a size of 64 bits (4 bytes)
- A double consists of:
  - 1 sign bit
  - 11 bit exponent with bias
  - 52 bit mantissa (or fraction, significand) with 1 bit implicitly stored



[2]

IEEE 754 Double-Precision Floating-Point Numbers (2)

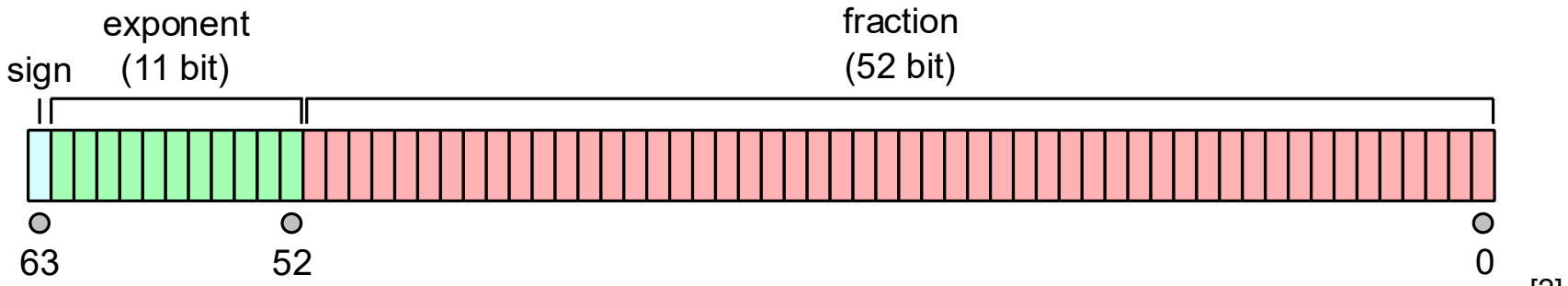
- What is the precision of this standard?
  - Quick answer: ~ 16 decimal digits
  - Longer answer:
    - Decimal string with at most 15 digits converted to a double printed as a string will be identical
    - Double converted to a string with at least 17 digits and converted back to a double will be identical



[2]

IEEE 754 Double-Precision Floating-Point Numbers (3)

- Floating-point numbers are not bijective with $\mathbb{R}$
  - Because the precision is limited by the size of a floating-point number
  - Rounding will occur when converting numbers to a floating-point number
    - Different rounding modes exist, default for binary floats is “round to nearest, ties to even”
- Precision is relative to the size, i.e., the amount of significant digits



[2]

Machine Epsilon ε

- The machine epsilon ε is the maximum relative error due to rounding
- Two different definitions exist, the interval machine epsilon is the more commonly used one
  - Also used in MATLAB (*eps*-function)
- For double:

$$\varepsilon = 2^{-52} \approx 2.22 \cdot 10^{-16}$$

- Interval machine epsilon is defined as the difference between 1 and the next larger floating-point number

Exercise 1:

- Implement the following sequence:

$$u_n = 111 - \frac{1130}{u_{n-1}} + \frac{3000}{u_{n-1}u_{n-2}}$$

- With the starting values of $u_0 = 2$ and $u_1 = -4$, calculate the value of u_n for 35 iterations

Exercise 1: Solution

- For u_4 a round-off error occurs
- Beginning with u_{16}, the value gets further away from 6
- In ~33 iterations the value has grown to 100
- The limit of the sequence depends on its initial values and is 6 with the initial values you used
- Since these values also cause the round-off error in the fourth value, the calculations are slightly off and the sequence converges on the limit of 100

n	Exact value
3	18.5
4	9.3783783783783784
5	7.8011527377521613833
6	7.1544144809752493535
11	6.2744385982163279138
12	6.2186957398023977883
16	6.0947394393336811283
17	6.0777223048472427363
18	6.0639403224998087553
19	6.0527217610161521934
20	6.0435521101892688678
21	6.0360318810818567800
22	6.0298473250239018567
23	6.0247496523668478987
30	6.0067860930312057585
31	6.0056486887714202679

[3]

Exercise 2:

- You get an offer from your bank with a new concept for a savings account where you deposit $e - 1$ €. The bank takes 1 € in charges from your account in the first year. In the second year, your capital is doubled and then 1 € in charges are subtracted. This continues in the third year; your capital is tripled and 1 € in charges are subtracted.
- Calculate how much your account will be worth in 25 years.
  - $e = 2.718281828459045235360287471 35\ldots\ldots$

Exercise 2: Solution

- Exact value after 25 years would be $0.0399\ €$
- With $\mathrm{a} = e - 1$ the sequence converges to zero.
  - When $a > e - 1$, then it converges to $+\infty$
  - When $a < e - 1$, then it converges to $-\infty$
- Rounding direction causes either convergence to $+\infty$ or $-\infty$ as e is transcendental and will be rounded when expressed as a floating-point number
- The accumulated error only starts causing converge to $+\infty$ after 17 iterations

NaN: Not a Number (1)

- Symbolic floating-point number, technically not a number (hence the name)
- Occurs when a number is undefined
- NaN is represented with the sign bit set to 0 or 1 and all bits of the biased exponent set to 1
  - Signaling NaN if the first bit of the trailing fraction is 0
  - Quiet NaN if the first bit of the trailing fraction is 1
- Signaling NaNs cause the invalid operation exception to be raised, most times they can then propagate as a quiet NaN
- Quiet NaNs do not raise exceptions, but cause every floating-point result of an operation to be a NaN as well

NaN: Not a Number (2)

- Typical occurrences are invalid operations such as, but not limited to:
  - The square root of a negative number
  - The logarithm of a negative number
  - $\sin^{-1}$ or $\cos^{-1}$ of a number greater than 1 or lesser than -1
    - All these examples result in a complex number, which cannot be represented with one floating-point number
- If something you programmed results in a NaN, check if any of these operations occur in your code

∞ and -0

- Other special values of floating-point numbers include infinity as well as a signed zero
  - Infinity can either be negative or positive
- Most operations with infinity are well-defined, division of two infinities will cause a NaN
- Overflow will cause an infinity to occur
- Signed zeros are represented differently but are equal when using comparison operators

Floating-Point Arithmetic

- Operations of the arithmetic on the real numbers only apply partially
  - Commutativity applies when the operations are rounded correctly
  - Associativity and distributivity are lost
- For many applications this loss does not cause significant problems, especially regarding multiplication
- Addition is more problematic, round-off errors can and will accumulate
- When multiple values are added, summing from lowest to highest is a good choice

Remarks on Number Representation (1)

- Working with floating-point numbers can cause unforeseen errors
  - If you checked all other error sources (typos, sign errors, etc.) check for round-off errors
- If values differ in about 16 orders of magnitude (for doubles) round-off errors are likely to occur
  - Example: Adding 1234567891234567 and either 1 or 0.9 will yield the same result as expected, but adding it and 0.4 will also yield the same result
- Behavior of these errors can differ depending on hardware

Remarks on Number Representation (2)

- Floating-point arithmetic for single- and double-precision is implemented on a hardware level in consumer-grade hardware, i.e., your computers
- Microcontrollers do not generally have floating-point units available, depends on the controller
  - Software-side floating-point arithmetic are often available, but are very slow
- The same problem arises for quadruple-precision floats
  - Double-precision is precise enough even for most applications in orbital mechanics

Motivation

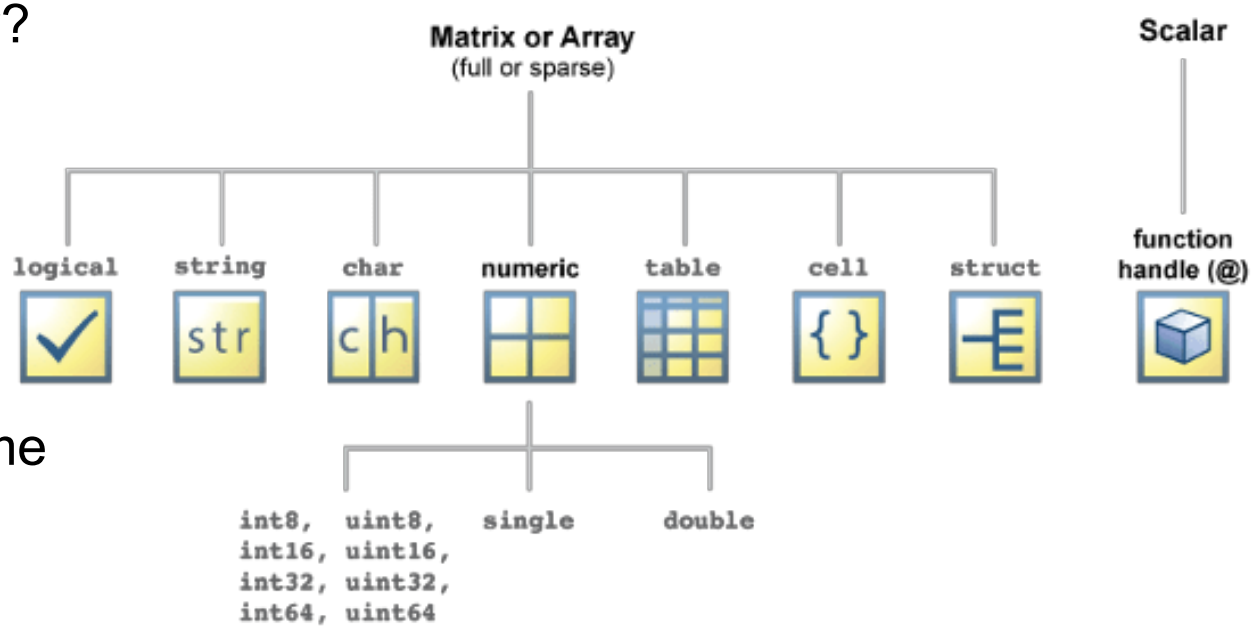
- When propagating an orbit, many functions are called for each time step
- Amount of time steps depends on the kind of propagation
  - Generally: The longer the propagation, the more time steps there are
- The performance of functions used in each time step is significant
  - Small improvements add up over a propagation
  - Also depends strongly upon the programming language used
- Where would you start looking for possible improvements?
  - Data access is a good place to start

Data Structures

- What are data structures?
  - Grouped primitive data types such as integers, floats, strings, booleans, etc.
  - Provides easier and more organized access to the data
- Available data structures depend strongly on the language used
- Most common structures include lists, arrays, tuples, queues/stacks, etc.
- In the following, the focus is on MATLAB data structures

Data Structures in MATLAB (1)

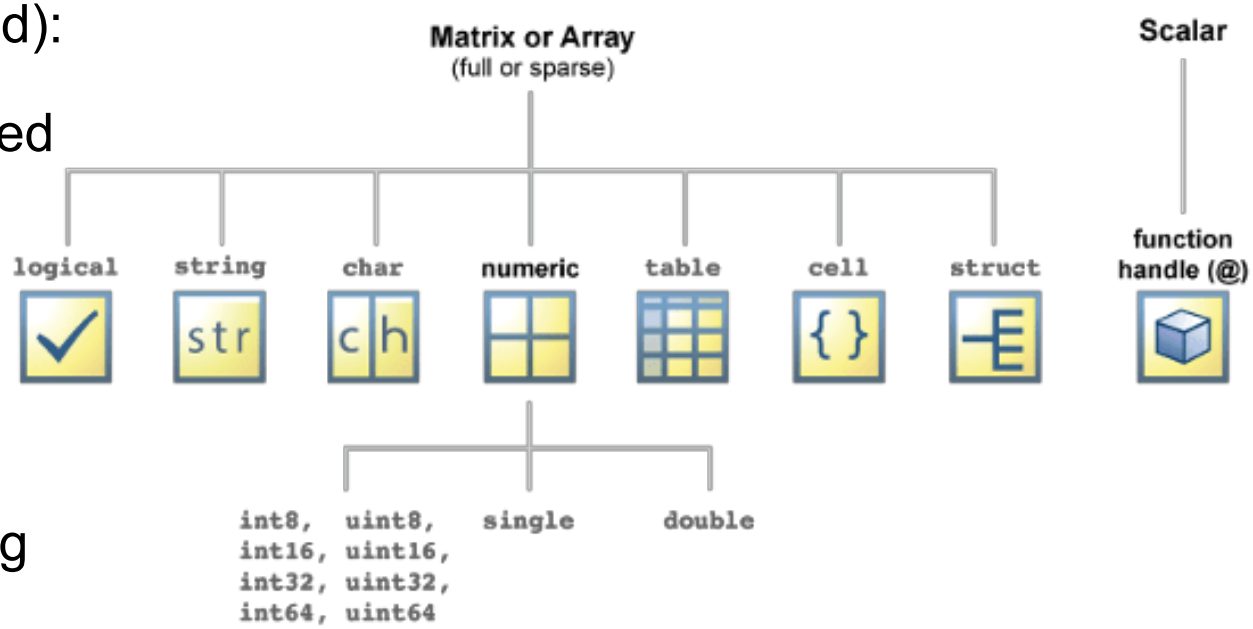
- Which MATLAB data structures do you know?
- MATLAB is class-based, data structures are called classes
- Typical data structures in MATLAB are:
  - Arrays: N-dimensional structure of the same data type, double by default
  - Matrices: 2-dimensional array, technically not different from arrays. Differentiation is important for built-in functions



[4]

Data Structures in MATLAB (2)

- Typical data structures in MATLAB are (cont'd):
  - Tables: mixed data types in column-oriented structure, rectangular shape
  - Cell arrays: mixed data types, stores arrays with varying sizes
  - Struct arrays: similar to cell arrays, differing in the syntax to access data

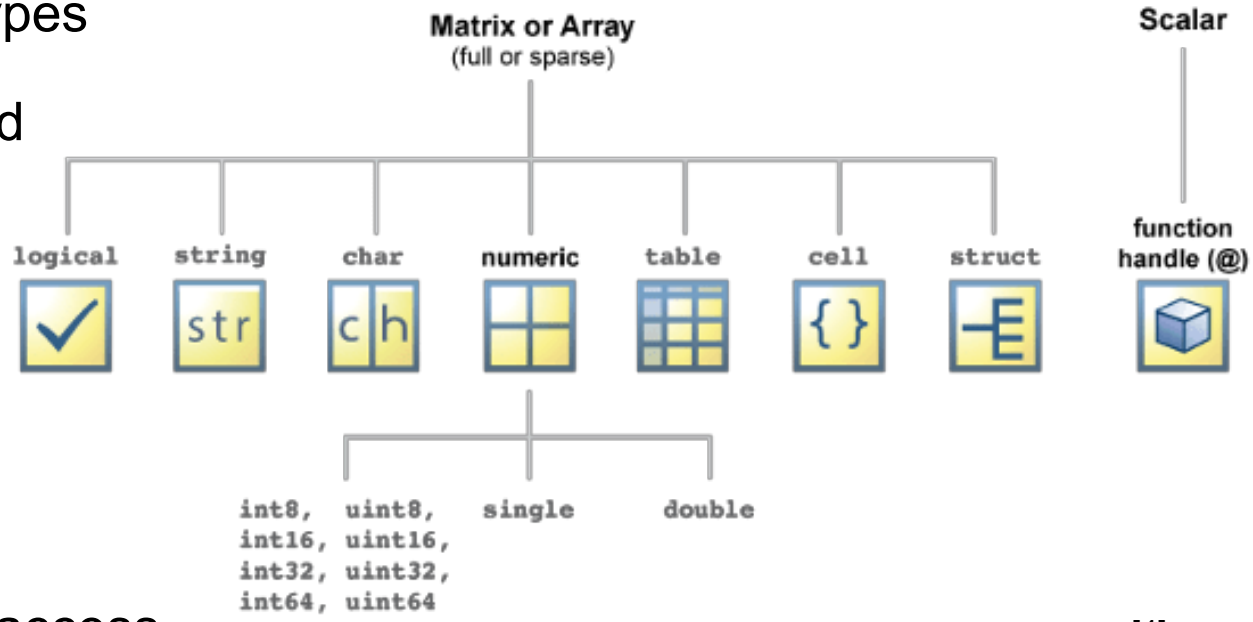


[4]

- Depending on the use case, some structures may have clear advantages over others

Data Structures in MATLAB (3)

- Syntax for data access differs between the types
- Tables and cells can be indexed, structs need the field names
  - Writing readable code can be difficult when field names are used, although it might be more comprehensible
- Another difference is in performance of data access
- Optimizing the use of data structures can improve performance of your code noticeably



[4]

Measuring Speed in MATLAB

- The simplest way to measure the speed of your code in MATLAB is to use the *tic* and *toc* functions
  - Simply add *tic* in the line where you want to start measuring and *toc* at the stop point
- Another way is to use the *timeit* function
  - For this, create a function handle to the function you want to measure with the input variables initialized before
  - Then, call the *timeit* function with the function handle as input
- Very helpful to find functions that might offer some improvements when optimized

Exercise 3:

- Create an array, a table and a cell array with dimension 100000x6 with random entries
  - *rand(dim1,dim2)*, *array2table, table2cell*
- Compare the three in writing and reading performance
  - Change the first 15000 entries of the 4. column in a for-loop
  - Read the first 15000 entries of the 4. column in a for-loop

Exercise 3:

- Tables are the slowest while arrays are the fastest
- When performance is of concern, use arrays as much as possible
- *tic* and *toc* are not very precise for code segments that take less than 1/10 s to execute
  - If you run your code again, the execution times can differ noticeably
  - Best way to test is to use *timeit,* measure execution time multiple times and calculate the mean
  - Needs functions though
- Arrays in structs are slower than arrays, but still quite fast

Other Performance Improvements (1)

- Using for-loops for operations over arrays is generally slow
  - MATLAB's strength are matrix operations
  - Vectorize as many calculations in MATLAB as possible
    - For cases like the previous exercise, array slices are faster than the for-loops
    - Vectorization might not be obvious immediately
- Functions are faster than scripts
  - Split your code into functions, especially for operations used more than once
  - Avoid nested functions, local functions are preferable

Other Performance Improvements (2)

- When possible, preallocate arrays instead of resizing it
  - Resizing always impacts performance, creating a new variable instead of resizing the existing one often is faster
  - Be careful of memory usage as many large arrays will fill the available memory quickly
- Avoid operations in loops that are independent of the loop
  - Every execution of an independent operation in the unnecessarily adds more time to the overall execution time
- Clearing variables helps with memory usage

Exercise 4:

- Use the array created in exercise 1 and calculate the cross product of the first three entries and the last three entries of each row
  1. With a for-loop
  2. With a vectorized approach (use MATLAB's built-in matrix functions)
- Compare the execution times for each approach

Exercise 4:

- The for-loop approach is slower than the vectorized approach
  - Execution is quite fast regardless of approach as this is a comparatively simple calculation
- Choosing the correct dimension can be tricky in more complex calculations
  - Comments in the code are quite useful in general, but especially here

Remarks

- Optimizing code is difficult
- Regardless of programming language, the execution time of for-loops and specifically nested for-loops increases with size
- When propagation an orbit associated arrays will be quite large
  - Computing parameters of interest such as access times or coverage will need significant time
- Deciding on which data structures to use as well as the likely size of each beforehand is a recommended approach

Image Index

[1] Montenbruck, Oliver, Gill, Eberhard. Satellite Orbits. 1st ed. Berlin: Springer Verlag; 2000.

[2] Wikipedia, *IEEE 754 Double Floating-Point Format*. https://en.wikipedia.org/wiki/File:IEEE_754_Double_Floating_Point_Format.svg#/media/File:IEEE_754_Double_Floating_Point_Format.svg

[3] Muller, Jean-Michel, et. al.. *Handbook of Floating-Point Arithmetic.* 1st ed. Berlin: Birkhäuser Verlag; 2010

[4] https://www.mathworks.com/help/matlab/matlab_prog/fundamental-matlab-classes.html